西南交通大學
Southwest Jiaotong University

《云计算技术》课程设计报告

设计题目: 云计算技术课程设计实验报告

学　　号: 2023112621,2023114337

成员姓名: 程泽凯，王晨羽

班　　级: 2023-04 班

指导教师: 戴朋林、丛荣　　老师

2026 年春季学期

《云计算技术》课程设计任务书

设计题目	云计算技术课程设计实验报告	成绩	
课程设计主要内容	本课程设计围绕云计算平台搭建与并行计算实践展开。第一部分基于华为云 CCE 容器平台，完成应用容器化、集群搭建、Deployment 与 Service 部署、Redis 持久化存储、ConfigMap 配置分离及 HPA 弹性伸缩验证。第二部分选择 Spark 大数据分析或 MPI 并行计算方向，完成环境部署、数据处理与性能分析。 课程设计以组队形式开展，每组 2-3 人，自行组队，报告中明确每个人的分工。报告使用老师提供的 LaTeX 模版，编译方式选 XeLaTeX。模版已写好第一至第六章的目录，尽量按照该目录撰写报告。		
指导教师评语	建议：从学生的工作态度、工作量、设计（论文）的创造性、学术性、实用性及书面表达能力等方面给出评价。 签名：　　年　　月　　日		

目录

一、学生信息与分工 **1**

二、实验目标与总体方案 **1**

三、华为云环境信息 **2**

四、第一部分：云计算平台搭建 **3**

4.1 任务 1：应用容器化 3

4.2 任务 2：CCE 集群搭建 4

4.3 任务 3：应用部署 5

4.4 任务 4：Redis 持久化存储 7

4.5 任务 5：ConfigMap Volume 挂载 9

4.6 任务 6：HPA 弹性伸缩 12

五、第二部分：Spark 大数据分析 **15**

5.1 A-0 环境部署 15

5.2 A-1 数据清洗 16

5.3 A-2 Spark SQL 统计分析 18

5.4 A-3 性能对比与 Amdahl 分析 21

六、附加题实践 **23**

6.1 附加题 1：Prometheus 与 Grafana 监控系统 23

6.2 附加题 2：CI/CD 流水线 25

6.3 附加题 3：边缘计算模拟：K3s + MQTT 28

七、关键文件与代码说明 **32**

八、总结与收获 **33**

参考文献 **34**

一、 学生信息与分工

表 **1**　课程设计基本信息

项目	内容
课程名称	云计算技术
姓名	程泽凯，王晨羽
学号	2023112621,2023114337
班级	2023-04 班
完成方式	小组协作完成
主要方向	CCE 容器平台搭建与 Spark 大数据分析

本课程设计围绕“本地容器化开发、华为云 Kubernetes 部署、并行计算作业运行和实验报告整理”展开。本组主要完成 Flask 后端 API、Nginx 前端页面、Redis 数据库联调、Docker 镜像构建、CCE YAML 编写、Redis 持久化验证、HPA 弹性伸缩验证、Spark 数据分析代码、性能对比分析以及附加题中的监控、CI/CD 和边缘计算模拟。

二、 实验目标与总体方案

本课程设计的目标是在华为云平台上完成一套可运行、可验证、可展示的云原生应用与并行计算实验。第一部分基于华为云 CCE 搭建 Kubernetes 平台，部署“Flask 后端 API + Redis 数据库 + Nginx 前端页面”的两层 Web 应用，并完成镜像仓库、对外暴露、配置分离、数据持久化和弹性伸缩验证。第二部分选择 Spark 大数据分析方向，在同一 CCE 集群上通过 Spark Operator 运行 PySpark 作业，完成数据清洗、统计分析和性能对比。

实验设计遵循从简单到复杂的顺序：首先在本地用 Docker Compose 验证应用功能，确保后端接口、前端页面和 Redis 访问没有逻辑错误；然后将镜像推送到 SWR，使云端集群可以拉取同一版本镜像；再使用 CCE 部署 Kubernetes 资源，通过 Deployment 管理副本，通过 Service 暴露服务，通过 ConfigMap 和 Secret 管理配置，通过 PVC 保存 Redis 数据；最后使用 HPA 观察负载变化下的自动扩缩容。这样的顺序可以减少排错范围，避免把应用代码问题、镜像问题和集群部署问题混在一起。

整体技术路线如表2所示。

表 **2**　课程设计总体技术路线

阶段	主要工作
本地开发	编写 Flask API、Nginx 静态页、Redis 访问逻辑和 Docker Compose 联调配置。
镜像管理	构建 backend/frontend 镜像，推送至华为云 SWR 私有镜像仓库。
CCE 部署	在 CCE 集群中部署 Deployment、Service、ConfigMap、Secret、PVC 和 HPA。
持久化与配置	使用 EVS PVC 保存 Redis 数据，使用 ConfigMap Volume 挂载 Nginx 配置文件。
并行计算	使用 Spark Operator 提交 PySpark 作业，完成清洗、SQL 查询和性能对比。
报告整理	整理关键命令、截图证据、实验结果、问题分析和总结。

三、 华为云环境信息

本实验涉及的华为云服务包括 SWR、CCE、ELB、OBS 和 EVS。SWR 用于存储本地构建的 Docker 镜像；CCE 用于运行 Kubernetes 工作负载；ELB 由 LoadBalancer 类型 Service 自动创建，用于对外暴露后端 API；OBS 用于存放 Spark 分析数据集；EVS 云硬盘通过 PVC 方式为 Redis 提供持久化存储。

表 **3**　华为云服务与用途

服务	在本实验中的作用
SWR	保存 backend 和 frontend 镜像，CCE 部署时从 SWR 拉取。
CCE	托管 Kubernetes 集群，运行 Pod、Deployment、Service、PVC 和 HPA。
ELB	为 LoadBalancer Service 提供公网访问入口。
OBS	存放 Spark 待分析数据集，通过 s3a 协议读取。
EVS	为 Redis PVC 提供云硬盘持久化能力。

实验环境截图应包含以下信息：Region、CCE 集群版本、Worker 节点规格、节点状态和 Kubernetes 版本。节点验证命令如下：

本次实验使用的主要本地目录为 `cloud-computing-final-project/` 和 `cloud-report-tex/`。前者保存 Dockerfile、Compose 文件、Kubernetes YAML、Spark 脚本和辅助数据；后者用于整理实验报告和截图。云端 Region 使用华为云华南区域，SWR 组织名为 `czk-course`。报告截图均尽量保留命令行提示符、资源名称、状态

列和返回结果，便于对应评分细则检查。

```
kubectl get nodes -o wide
```

截图要求为所有 Worker 节点状态均为 Ready，并且输出中包含 VERSION 列。

四、 第一部分：云计算平台搭建

4.1 任务 1：应用容器化

本任务从本地应用开始实施。实验中先编写 Flask 后端服务，提供 /api/ping 健康检查接口和 /api/visit Redis 写入接口；随后编写 Nginx 前端首页，在页面中展示程泽凯、王晨羽及学号 2023112621,2023114337，用于验收时识别。完成代码后，使用 Docker Compose 同时启动 backend、frontend 和 Redis，先在本地确认接口、页面和 Redis 访问逻辑均正常，再进入镜像构建和推送环节。

后端 requirements.txt 中除 Flask 和 Redis 客户端外，额外加入 requests 包，满足“加入至少 1 个自选 Python 包”的要求。后端 Dockerfile 保留 builder/runtime 多阶段构建结构，并通过 PIP_INDEX_URL 默认使用清华 PyPI 镜像源；前端 Dockerfile 使用 Nginx 镜像托管静态首页。构建镜像时 Docker Desktop 曾出现 BuildKit 服务异常，处理方式是重启 Docker Desktop、执行 wsl --shutdown 并清理构建缓存。推送 SWR 时还遇到 BuildKit provenance manifest 无法被 SWR 解析的问题，因此重新使用 docker build --provenance=false 构建镜像，再执行 docker tag 和 docker push。最终 SWR 组织 czk-course 下出现 backend 与 frontend 两个镜像，Tag 均为 v1。

多阶段构建的作用是将依赖安装和运行环境分离，减少最终镜像中不必要的构建缓存和临时文件。后端镜像在 builder 阶段安装 Python 依赖，在 runtime 阶段只复制运行所需文件；前端镜像则只保留 Nginx 与静态页面。这样做既符合课程要求，也更接近实际云端交付流程。Docker Compose 联调阶段，我重点观察了两个现象：一是 /api/ping 能稳定返回 status=ok，二是后端日志能记录请求来源，说明前后端和后端容器自身网络均可正常工作。

Listing 1: 本地容器联调命令

```
docker compose up --build
curl http://localhost:5000/api/ping
curl http://localhost:5000/api/visit
```

预期 /api/ping 返回如下 JSON：

```
{"status":"ok"}
```

本任务的验证依据包括：`docker compose up --build -d` 本地联调输出、`docker compose ps` 中三个容器均处于 Up 状态、前端反向代理访问 `/api/ping` 返回 `{"status":"ok"}`、后端日志中出现 `GET /api/ping` 请求记录，以及 SWR 控制台中 backend/frontend 镜像列表。

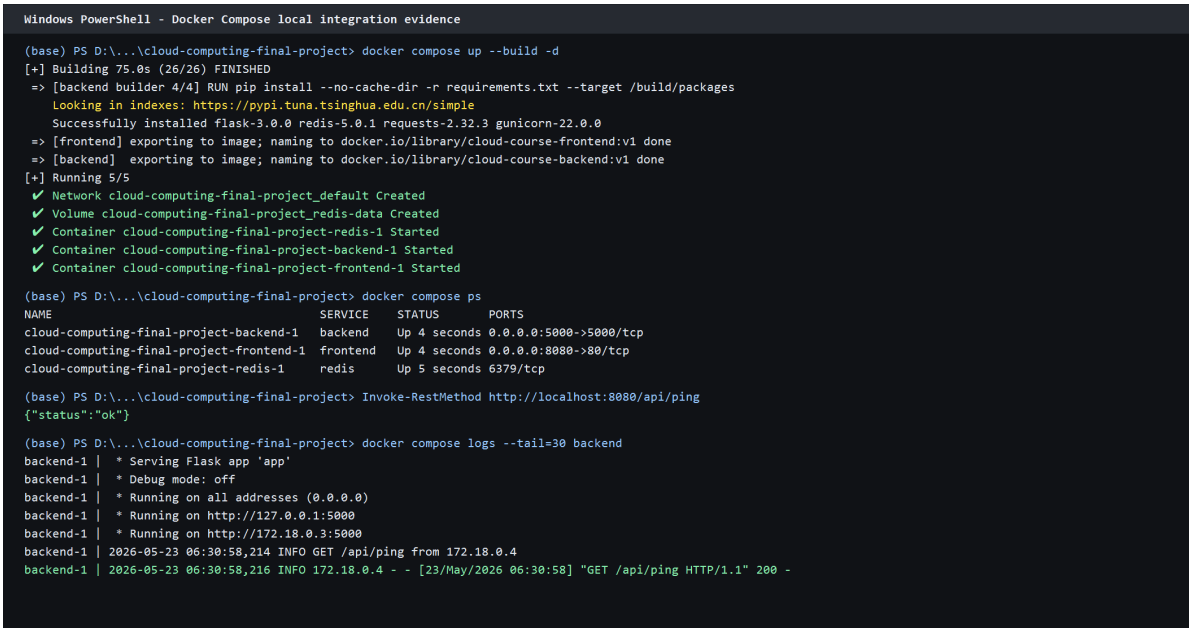


图 1　**Docker Compose** 本地联调运行状态与后端日志



图 2　**SWR** 控制台中 **backend** 与 **frontend** 镜像列表

4.2 任务 2：CCE 集群搭建

集群部分在华为云 CCE 控制台完成。实验中创建了 2 个 Worker 节点的 Kubernetes 集群，Master 节点由华为云托管。初次使用 `kubectl` 时，kubeconfig 曾指向本机 `127.0.0.1` 临时端口，导致 `kubectl get nodes` 无法连接 API Server；随后更换为 CCE 控制台下载的公网 kubeconfig，并在 PowerShell 中设置 `KUBECONFIG` 环境变量，重新验证集群节点。

Listing 2: CCE 节点状态验证

```
kubectl get nodes -o wide
```

验证结果显示两个 Worker 节点 IP 分别为 `192.168.0.23` 和 `192.168.0.59`，状态均为 `Ready`，Kubernetes 版本为 `v1.34.3-r0-34.0.12.3`，高于任务要求的 1.27。至此，后续 Deployment、Service、PVC 和 HPA 实验具备运行基础。

节点验证不仅检查 Ready 状态，也需要关注 VERSION、OS-IMAGE 和 CONTAINER-RUNTIME 等信息。本实验节点操作系统为 Ubuntu 22.04.5 LTS，容器运行时为 containerd，说明集群采用当前主流的 Kubernetes 运行方式。由于后续 Redis PVC、LoadBalancer 和 HPA 都依赖 CCE 的云插件和节点资源，节点状态截图是后续所有任务的基础证据。



图 **3** **CCE** 节点状态验证

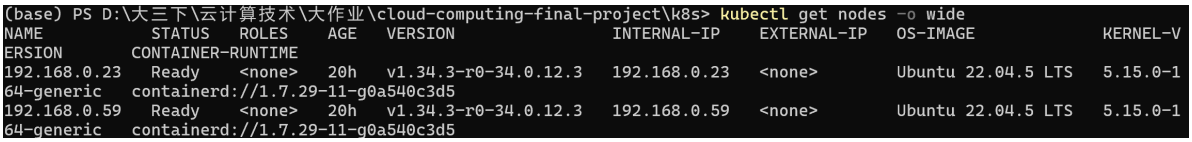


图 **4** 本地终端验证 **CCE Worker** 节点 **Ready**

4.3 任务 3：应用部署

应用部署阶段将任务 1 中推送到 SWR 的镜像部署到 CCE 集群。实验中先创建 Redis 密码 Secret 和 Redis 地址 ConfigMap，再创建 Redis PVC、Redis Deployment、后端 Deployment 和 Service。后端镜像使用 `swr.cn-south-1.myhuaweicloud.com/czk-course/backend:v1`，前端镜像使用 `swr.cn-south-1.myhuaweicloud.com/czk-course/frontend:v1`。

后端 Deployment 的副本数设置为 2，容器端口为 5000，并配置 CPU、内存的 requests 和 limits；Redis Deployment 的副本数为 1，内存限制不超过 512Mi。Redis 主机名和端口通过 ConfigMap 注入，Redis 密码通过 Secret 注入，避免在 Deployment 中直接写明文密码。部署过程中，后端 Pod 曾因为 SWR 私有镜像拉取权限不足出现 401 Unauthorized，解决方法是在集群中创建 `swr-secret`，并在 backend/frontend Deployment 中配置 `imagePullSecrets`。

Listing 3: 应用部署命令

```
kubectl apply -f secret.yaml
kubectl apply -f configmap.yaml
kubectl apply -f redis-pvc.yaml
kubectl apply -f redis-deployment.yaml
kubectl apply -f backend-deployment.yaml
kubectl apply -f frontend-configmap.yaml
kubectl apply -f frontend-deployment.yaml
kubectl apply -f services.yaml
```

部署完成后，使用如下命令验证 Pod 状态和公网访问：

```
kubectl get pods
kubectl get svc backend-svc
curl http://<ELB_IP>/api/ping
```

后端 Service 最初只配置 `kubernetes.io/elb.class: union` 时，`EXTERNAL-IP` 长时间为 `<pending>`。查看 Service 事件后发现，华为云 CCE 自动创建公网 ELB 还需要 `kubernetes.io/elb.autocreate` 注解。因此实验中补充自动创建公网 ELB 的注解，删除并重建 `backend-svc`，最终获得公网 IP `110.41.67.203`。通过 PowerShell 访问 `http://110.41.67.203/api/ping`，返回 HTTP 200 和 `{"status":"ok"}`，后端日志同步显示 `GET /api/ping` 请求，说明后端已通过 ELB 正常暴露。

在资源组织上，后端 Service 采用 LoadBalancer 类型用于公网访问，Redis Service 采用 ClusterIP 类型，仅允许集群内部访问。这样的设计符合最小暴露原则：外部用户只需要访问后端 API，不需要直接访问 Redis 数据库。后端通过 ConfigMap 获取 `REDIS_HOST=redis-svc` 和 `REDIS_PORT=6379`，通过 Secret 获取 Redis 密码，从而把业务镜像和运行环境配置解耦。截图中 backend 两个 Pod 分布在不同节点上，Redis 一个 Pod 独立运行，能够体现 Deployment 副本和 Service 访问入口均已生效。

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get pods -o wide
NAME                     READY   STATUS    RESTARTS   AGE     IP             NODE            NOMINATED NODE   READINESS
 GATES
backend-857bc7464b-jnpz8   1/1     Running   0          8m27s   10.244.0.11    192.168.0.23   <none>           <none>
backend-857bc7464b-td4gv   1/1     Running   0          8m11s   10.244.0.139   192.168.0.59   <none>           <none>
redis-7cb6d76c74-x4ksn     1/1     Running   0          10m     10.244.0.10    192.168.0.23   <none>           <none>
```

图 **5**　后端与 **Redis Pod** 均处于 **Running** 状态

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get svc backend-svc redis-svc
NAME          TYPE           CLUSTER-IP     EXTERNAL-IP                  PORT(S)        AGE
backend-svc   LoadBalancer   10.96.185.34   110.41.67.203,192.168.0.18   80:31072/TCP   6m8s
redis-svc     ClusterIP      10.96.163.10   <none>                       6379/TCP       16m
```

图 **6**　后端 **LoadBalancer Service** 与 **Redis ClusterIP Service**



图 7　通过 **ELB** 公网 **IP** 访问 **/api/ping** 返回 **200**

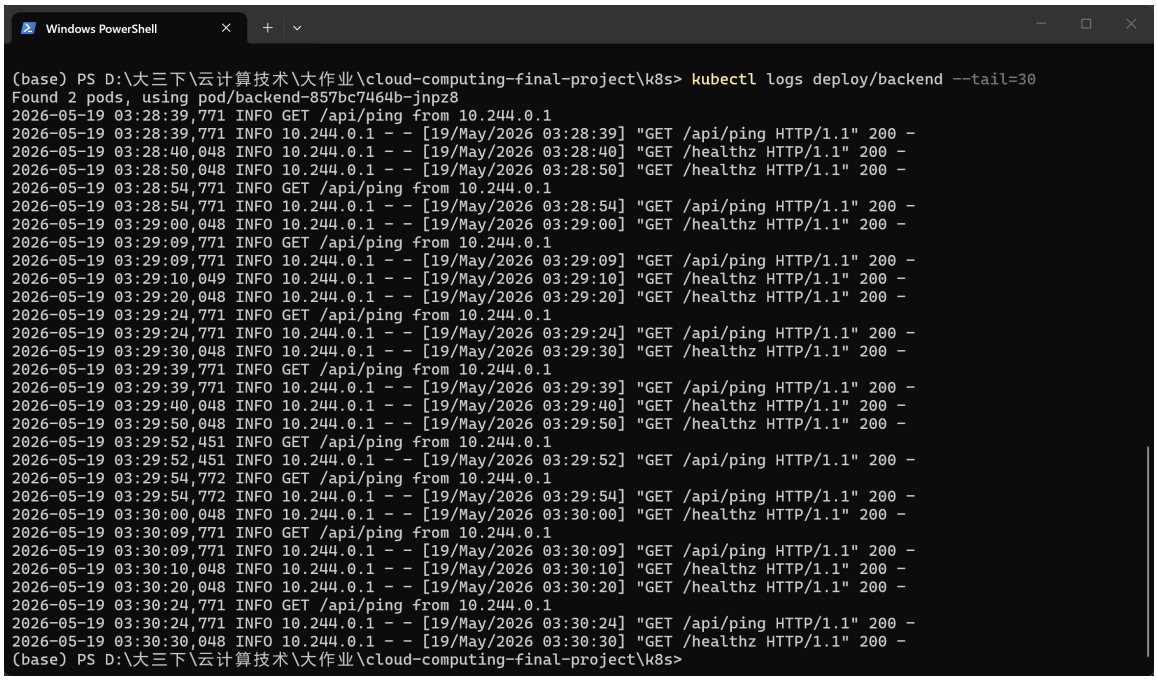


图 **8**　后端日志显示收到 **/api/ping** 请求

4.4 任务 4：Redis 持久化存储

Redis 持久化部分使用华为云 EVS 云硬盘。实验中先创建 `redis-data-pvc`，设置 `storageClassName` 为 `csi-disk`，申请容量为 10Gi；随后在 Redis Deployment 中将该 PVC 挂载到容器 `/data` 目录，并启动 Redis AOF 持久化。这样 Redis 的数据文件不会随着 Pod 删除而丢失。

Listing 4: Redis PVC 验证命令

```
kubectl get pvc
kubectl exec -it <redis-pod-name> -- redis-cli -a <password> SET testkey "hello"
kubectl exec -it <redis-pod-name> -- redis-cli -a <password> GET testkey
kubectl delete pod <redis-pod-name>
kubectl exec -it <new-redis-pod-name> -- redis-cli -a <password> GET testkey
```

验证时先执行 `kubectl get pvc`，确认 `redis-data-pvc` 状态为 `Bound`，StorageClass 为 `csi-disk`。随后进入 Redis Pod，执行 `SET testkey "hello"` 写入测试数据，再执行 `GET testkey` 得到 `"hello"`。接着删除当前 Redis Pod，由 Deployment 自动重建新 Pod。新 Pod 运行后再次执行 `GET testkey`，仍返回 `"hello"`，证明数据保存在 PVC 对应的云硬盘中，而不是保存在旧 Pod 的临时文件系统中。

这个实验能区分“Pod 重建”和“数据重建”的区别。Pod 是临时运行单元，删除后其容器文件系统会消失；PVC 绑定的是独立云硬盘，生命周期可以长于 Pod。因此只要 Redis 的数据目录挂载到 PVC，并开启 AOF 或 RDB 持久化，数据就不会因为 Pod 名称变化而丢失。本实验中旧 Redis Pod 被删除后，新 Pod 名称发生变化，但 `testkey` 仍能读取，说明持久化路径和 PVC 绑定关系正确。

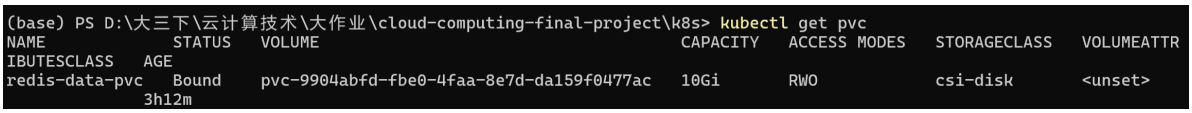


图 9　**Redis PVC 状态为 Bound，StorageClass 为 csi-disk**

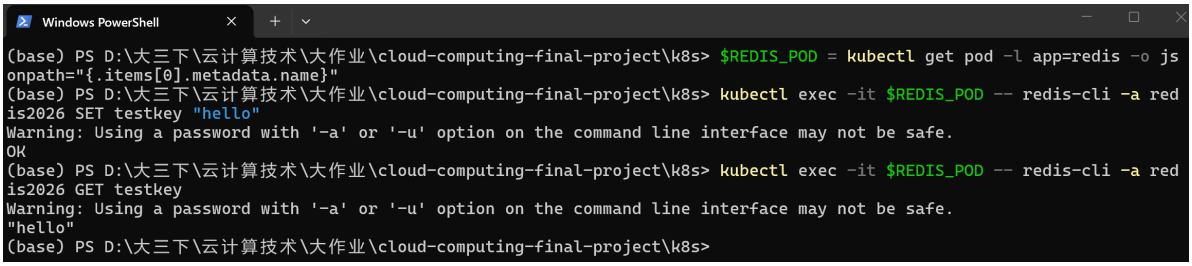


图 10　**Redis 写入并读取 `testkey=hello`**

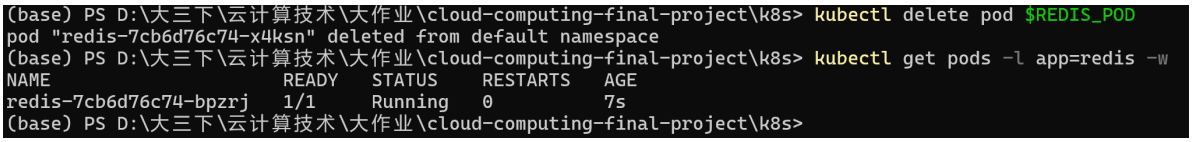


图 11　删除 **Redis Pod** 后由 **Deployment** 自动重建

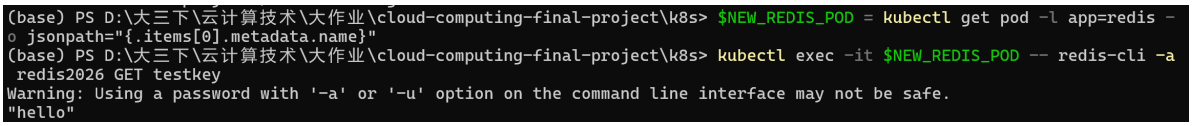


图 12　**Redis Pod** 重建后仍可读取 `testkey=hello`

4.5 任务 5：ConfigMap Volume 挂载

本任务将前端 Nginx 反向代理配置改为 ConfigMap Volume 挂载。实验中先创建 `frontend-nginx-config` ConfigMap，在 `data.default.conf` 中写入完整 Nginx 配置，其中 upstream 指向后端 Service。随后修改 frontend Deployment，将该 ConfigMap 挂载到容器内 `/etc/nginx/conf.d/default.conf`，并通过 `subPath` 指定挂载单个配置文件。

Listing 5: ConfigMap Volume 验证命令

```
kubectl apply -f frontend-configmap.yaml
kubectl rollout restart deployment/frontend
kubectl exec -it <frontend-pod-name> -- cat /etc/nginx/conf.d/default.conf
```

为验证 ConfigMap Volume 是否真正生效，实验中将 upstream 端口从 `backend-svc:5000` 临时修改为 `backend-svc:5001`，执行 `kubectl apply` 并滚动重启前端 Deployment。进入新的前端 Pod 后，读取 `/etc/nginx/conf.d/default.conf`，可以看到配置文件中已经变为 `server backend-svc:5001;`。截图完成后，再将端口恢复为 `5000` 并重新发布前端，避免影响后续访问。Volume 挂载方式适合 Nginx 配置、证书、应用配置文件等文件型配置，程序可以像读取普通文件一样读取配置内容；`envFrom` 更适合 Redis 地址、端口、运行环境等少量键值型配置，应用在启动时从环境变量中读取。

与环境变量相比，Volume 挂载更适合完整配置文件，因为 Nginx 本身就是读取 `/etc/nginx/conf.d/default.conf` 文件启动服务。如果使用环境变量，还需要额外模板渲染或入口脚本才能生成配置文件，复杂度更高。ConfigMap Volume 的另一个优点是配置内容可以直接通过 `kubectl get configmap -o yaml` 查看，也可以通过 `kubectl exec` 进入容器检查最终落盘文件，便于验收和排错。

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl apply -f frontend-configmap.yaml
configmap/frontend-nginx-config created
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl apply -f frontend-deployment.yaml
deployment.apps/frontend created
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get configmap frontend-nginx-config -o
yaml
apiVersion: v1
data:
  default.conf: |
    upstream backend_upstream {
        server backend-svc:5000;
    }

    server {
        listen 80;
        server_name _;

        root /usr/share/nginx/html;
        index index.html;

        location / {
            try_files $uri $uri/ /index.html;
        }

        location /api/ {
            proxy_pass http://backend_upstream/api/;
            proxy_http_version 1.1;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }
    }
kind: ConfigMap
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","data":{"default.conf":"upstream backend_upstream {\n    server backend-svc:5000;\n}\n\nserver
{\n    listen 80;\n    server_name _;\n\n    root /usr/share/nginx/html;\n    index index.html;\n\n    location / {\n
    try_files $uri $uri/ /index.html;\n    }\n\n    location /api/ {\n        proxy_pass http://backend_upstream/api/;\
n        proxy_http_version 1.1;\n        proxy_set_header Host $host;\n        proxy_set_header X-Real-IP $remote_addr;
\n        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;\n        proxy_set_header X-Forwarded-Proto $sche
me;\n    }\n}\n"},"kind":"ConfigMap","metadata":{"annotations":{},"name":"frontend-nginx-config","namespace":"default"}}
```

图 **13**　**ConfigMap** 中保存完整 **Nginx** 反向代理配置



图 **14**　前端 **Deployment** 通过 **Volume** 挂载 **ConfigMap**

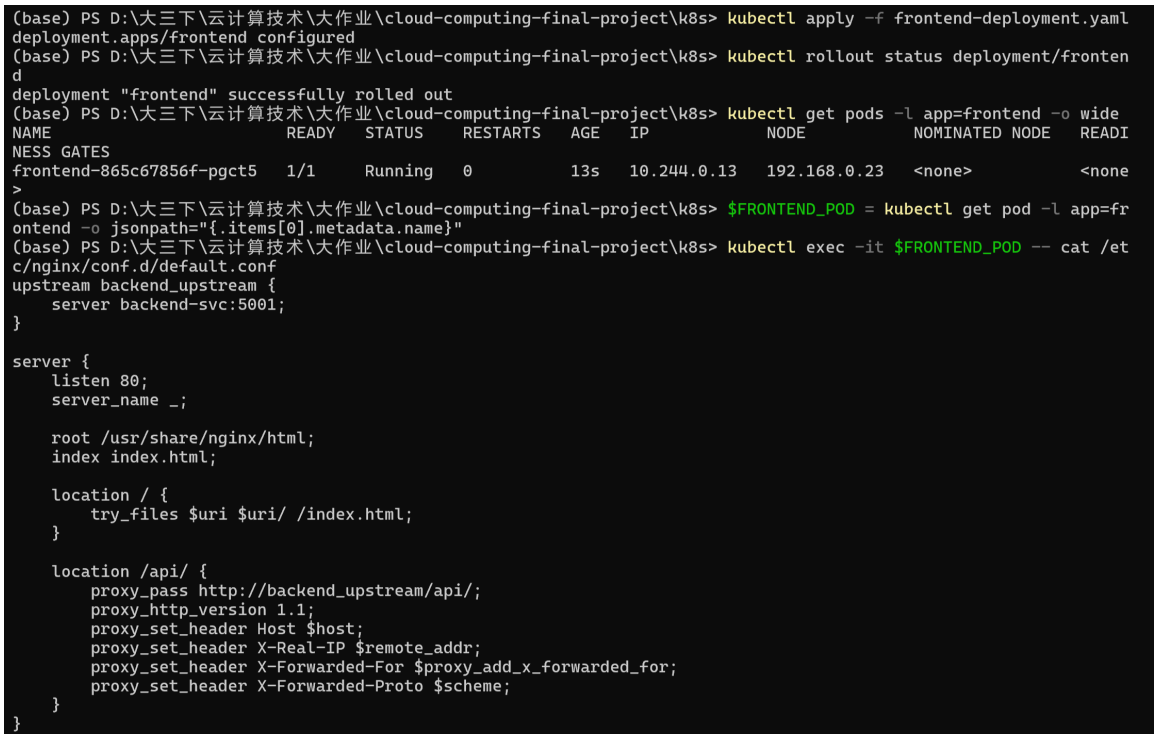


图 **15**　修改 **ConfigMap** 端口后在前端 **Pod** 内验证配置文件已更新

4.6 任务 6：HPA 弹性伸缩

HPA 实验用于验证后端服务能否根据 CPU 利用率自动扩缩容。实验中先安装并检查 metrics-server，使 `kubectl top nodes` 和 `kubectl top pods` 能返回 CPU、内存指标；然后创建 `backend-hpa`，设置 `minReplicas=1`、`maxReplicas=4`、CPU 目标利用率为 60%。创建 HPA 后，先将 backend 缩到 1 个副本，便于观察后续扩容过程。

Listing 6: HPA 验证命令

```
kubectl top nodes
kubectl apply -f hpa.yaml
kubectl get hpa
kubectl get pods -w
```

本机没有安装 `ab` 工具，因此实验中使用 PowerShell `Start-Job` 并发脚本替代压测。压测脚本向 ELB 公网地址 `http://110.41.67.203/api/ping` 并发发送请求，同时在另一个终端执行 `kubectl get pods -l app=backend -w` 和 `kubectl get hpa` 观察副本变化。

```
1..40 | ForEach-Object {
 Start-Job {
  for ($i = 0; $i -lt 500; $i++) {
   Invoke-WebRequest -UseBasicParsing http://110.41.67.203/api/ping | Out-Null
  }
```

```
  }
}
```

由于 /api/ping 接口逻辑很轻，初次压测时 CPU 利用率没有明显上升。为完成 HPA 验证，实验中临时降低 backend 的 requests.cpu，使 CPU 利用率百分比能反映轻量请求带来的压力。压测期间 HPA 最高观察到 600%/60%，backend 副本数由 1 扩展到 4。停止压测后等待冷却窗口，HPA 最终将副本缩回 1。扩容延迟主要来自 metrics-server 的采集周期和 HPA 控制器的评估间隔；缩容冷却时间用于防止负载短时间波动时频繁扩缩容；HPA 的价值在于高负载时自动增加副本保障可用性，低负载时减少副本降低云资源成本。

HPA 的百分比并不是直接使用节点 CPU 百分比，而是根据 Pod 当前 CPU 使用量与容器 requests.cpu 的比例计算。因此在轻量接口压测场景下，如果 request 设置较大，HPA 可能认为负载仍然很低。实验中先确认 metrics-server 可用，再观察 kubectl get hpa 中 TARGETS 字段变化，最后结合 kubectl get pods -l app=backend -w 记录副本数变化。扩容到 4 个副本后，说明 HPA 控制器、指标采集、Deployment 副本调整三部分链路均已打通；停止压测后缩回 1 个副本，说明缩容策略也能正常工作。

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl top nodes
NAME           CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
192.168.0.23   75m          3%       995Mi           16%
192.168.0.59   74m          3%       1052Mi          17%
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl top pods
NAME                        CPU(cores)   MEMORY(bytes)
backend-857bc7464b-td4gv    1m           33Mi
frontend-5d6b9cb557-blf2z   0m           2Mi
redis-7cb6d76c74-bpzrj      4m           2Mi
```

图 **16**　**metrics-server** 可用，**kubectl top nodes/pods** 返回资源指标

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> 1..40 | ForEach-Object {
>>   Start-Job {
>>     for ($i = 0; $i -lt 500; $i++) {
>>       try {
>>         Invoke-WebRequest -UseBasicParsing http://110.41.67.203/api/ping | Out-Null
>>       } catch {}
>>     }
>>   }
>> }

Id     Name            PSJobTypeName   State         HasMoreData     Location             Command
--     ----            -------------   -----         -----------     --------             -------
9      Job9            BackgroundJob   Running       True            localhost            ...
11     Job11           BackgroundJob   Running       True            localhost            ...
13     Job13           BackgroundJob   Running       True            localhost            ...
15     Job15           BackgroundJob   Running       True            localhost            ...
17     Job17           BackgroundJob   Running       True            localhost            ...
19     Job19           BackgroundJob   Running       True            localhost            ...
21     Job21           BackgroundJob   Running       True            localhost            ...
```

图 **17**　使用 **PowerShell** 并发请求脚本替代 **ab** 发起压测

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME          REFERENCE              TARGETS        MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa   Deployment/backend   cpu: 10%/60%   1         4         1          9m35s
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl top pods
NAME                        CPU(cores)   MEMORY(bytes)
backend-857bc7464b-td4gv    1m           33Mi
frontend-5d6b9cb557-blf2z   0m           2Mi
redis-7cb6d76c74-bpzrj      3m           2Mi
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get pods -l app=backend -w
NAME                       READY   STATUS    RESTARTS   AGE
backend-857bc7464b-td4gv   1/1     Running   0          3h41m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl top pods
NAME                        CPU(cores)   MEMORY(bytes)
backend-857bc7464b-td4gv    1m           33Mi
frontend-5d6b9cb557-blf2z   0m           2Mi
redis-7cb6d76c74-bpzrj      3m           2Mi
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME          REFERENCE              TARGETS        MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa   Deployment/backend   cpu: 10%/60%   1         4         1          10m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME          REFERENCE              TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa   Deployment/backend   cpu: 600%/60%   1         4         4          15m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl top pods
NAME                        CPU(cores)   MEMORY(bytes)
backend-5c848d6c6d-94xzb    5m           33Mi
backend-5c848d6c6d-c5kpx    5m           32Mi
backend-5c848d6c6d-f6v4w    5m           32Mi
backend-5c848d6c6d-jtxwt    5m           32Mi
frontend-5d6b9cb557-blf2z   0m           2Mi
redis-7cb6d76c74-bpzrj      4m           2Mi
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get pods -l app=backend -w
NAME                       READY   STATUS    RESTARTS   AGE
backend-5c848d6c6d-94xzb   1/1     Running   0          4m1s
backend-5c848d6c6d-c5kpx   1/1     Running   0          4m11s
backend-5c848d6c6d-f6v4w   1/1     Running   0          2m1s
backend-5c848d6c6d-jtxwt   1/1     Running   0          2m1s
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl top pods
NAME                        CPU(cores)   MEMORY(bytes)
backend-5c848d6c6d-94xzb    5m           33Mi
backend-5c848d6c6d-c5kpx    5m           32Mi
backend-5c848d6c6d-f6v4w    5m           32Mi
backend-5c848d6c6d-jtxwt    5m           32Mi
frontend-5d6b9cb557-blf2z   0m           2Mi
```

图 **18**　**HPA** 在压测期间将 **backend** 副本扩展到 **4** 个

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME          REFERENCE              TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa   Deployment/backend   cpu: 500%/60%   1         4         4          25m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME          REFERENCE              TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa   Deployment/backend   cpu: 100%/60%   1         4         2          26m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME          REFERENCE              TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa   Deployment/backend   cpu: 100%/60%   1         4         4          27m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME          REFERENCE              TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa   Deployment/backend   cpu: 100%/60%   1         4         4          27m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME          REFERENCE              TARGETS        MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa   Deployment/backend   cpu: 10%/60%   1         4         4          28m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME          REFERENCE              TARGETS        MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa   Deployment/backend   cpu: 10%/60%   1         4         4          29m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME          REFERENCE              TARGETS        MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa   Deployment/backend   cpu: 10%/60%   1         4         1          32m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl top pods
NAME                        CPU(cores)   MEMORY(bytes)
backend-857bc7464b-9lrsj    1m           32Mi
frontend-5d6b9cb557-blf2z   0m           2Mi
redis-7cb6d76c74-bpzrj      3m           2Mi
```

图 **19**　压测停止后 **HPA** 缩容回 **1** 个副本

五、 第二部分：Spark 大数据分析

5.1 A-0 环境部署

本报告第二部分选择方向 A：Spark 大数据分析。教师下发离线资源包后，本实验优先采用课程提供的资源完成环境部署：先将 spark-operator-2.5.0.tar 和 pyspark-v9.tar 导入本地 Docker，再重新打 Tag 推送到本人华为云 SWR 组织 czk-course。最终 CCE 集群拉取的控制器镜像为 swr.cn-south-1.myhuaweicloud.com/czk-course/spark-operator:2.5.0，PySpark 作业镜像为 swr.cn-south-1.myhuaweicloud.com/czk-course/pyspark:v9。其中 pyspark:v9 内部 Spark 版本为 3.4.3，并自带 /opt/spark/examples/src/main/python/wordcount.py 示例程序和样例输入文件，适合用于 A-0 环境连通性验证。

Listing 7: 教师离线包镜像导入、推送与作业提交命令

```
docker load -i spark/spark-operator-2.5.0.tar
docker load -i spark/pyspark-v9.tar
docker tag ghcr.io/kubeflow/spark-operator/controller:2.5.0 \
  swr.cn-south-1.myhuaweicloud.com/czk-course/spark-operator:2.5.0
docker tag swr.cn-east-3.myhuaweicloud.com/cloud-course-2025212245/pyspark:v9 \
  swr.cn-south-1.myhuaweicloud.com/czk-course/pyspark:v9
docker push swr.cn-south-1.myhuaweicloud.com/czk-course/spark-operator:2.5.0
docker push swr.cn-south-1.myhuaweicloud.com/czk-course/pyspark:v9

helm upgrade --install spark-op ./spark-operator/ \
  -n spark-operator --create-namespace \
  -f spark-operator-teacher-values.yaml

kubectl apply -f sparkapplication-wordcount-teacher.yaml
kubectl get pods -n default
kubectl logs <driver-pod-name>
```

sparkapplication-wordcount-teacher.yaml 采用 cluster 模式运行 Python 作业，镜像为本人 SWR 中的 pyspark:v9，Spark 版本为 3.4.3，主程序为镜像内置的 WordCount 示例。作业参数设置为 2 个 Executor，每个 Executor 内存为 1g。Driver Pod 使用 spark-op-spark-operator-spark 这个 ServiceAccount 提交作业，两个 Executor 分别由 Spark Operator 自动创建。由于 SWR 镜像仓库未设置为公开，本实验在 spark-operator 和 default 命名空间创建 swr-secret，并在 Operator Helm values 与 SparkApplication 中配置 imagePullSecrets，保证 CCE 节点能够认证拉取课程镜像。

实际部署过程中主要处理了三个问题。第一，课程镜像体积较大，推送 spark-operator:2.5.0 和 pyspark:v9 用时较长，需要确认 Docker 登录凭证有

效后再 push。第二，Operator 切换到本人 SWR 镜像后曾出现 401 Unauthorized，原因是 CCE 节点匿名拉取私有镜像失败，创建 swr-secret 后问题解决，控制器 Pod 成功进入 Running，镜像 ID 指向本人 SWR 中的 spark-operator:2.5.0。第三，课程 CCE 集群资源较紧，最初 Driver Pod 曾因 Insufficient cpu 处于 Pending，因此在 YAML 中设置 spark.kubernetes.driver.request.cores=10m 和 spark.kubernetes.executor.request.cores=10m。最终 pyspark-wordcount-teacher 作业创建了 1 个 Driver Pod 和 2 个 Executor Pod，两个 Executor 分别调度到两个 Worker 节点运行；作业结束后 SparkApplication 状态为 COMPLETED，Driver 日志中出现 WordCount 统计结果和 exitCode 0，说明教师离线资源包版本验证通过。

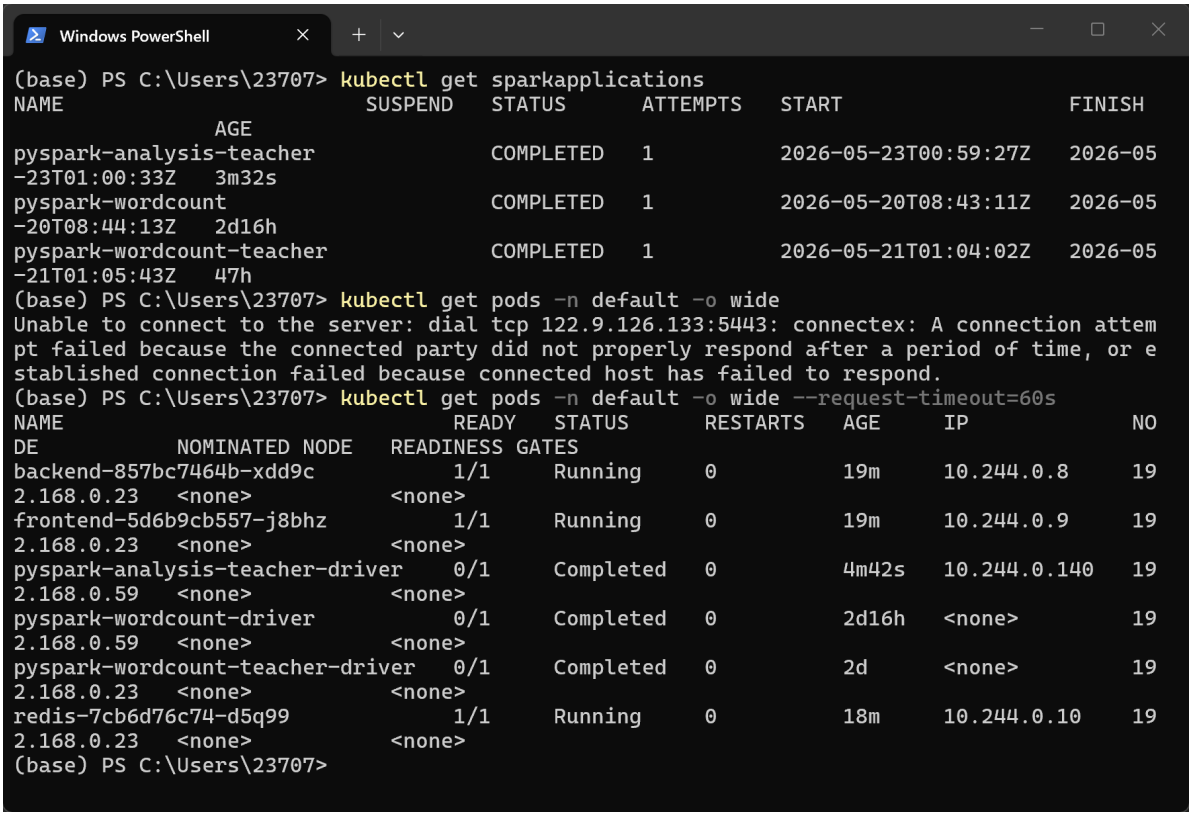


图 **20**　**SparkApplication** 与 **Driver/Executor Pod** 运行状态

5.2 A-1 数据清洗

教师新发的正式数据文件为 douban_movies.csv，大小约 39 MB，包含影片名称、年份、评分、评分人数、类型、国家、导演、收藏数和简介等字段。为保证 CCE 上的输入文件、分析脚本和 Spark 运行环境可复现，我将该数据文件与更新后的 analysis.py 一同构建到镜像 pyspark-analysis:douban-v4，推送到 SWR 后通过 SparkApplication 启动 1 个 Driver 和 2 个 Executor 执行正式分析作业。

首次运行时出现了一个容易被忽略但会严重影响结论的问题：summary 字段包含带双引号的跨行文本，若按默认 CSV 参数读取，Spark 会将简介内部换行误认为新记录，

错误地产生 117,156 行，并出现评分最大值 2002.0 等明显异常。定位原因后，我在读取阶段启用 multiLine 并明确设置引号与转义规则，重新运行后记录总数恢复为 67,132 行，字段类型和本地核对结果一致。因此后续统计均以修正后的正式数据运行结果为准。

Listing 8: 正式数据读取与清洗代码

```
raw = (spark.read.option("header", True)
      .option("inferSchema", True)
      .option("multiLine", True)
      .option("quote", '"')
      .option("escape", '"')
      .csv(input_path))
prepared = (raw.withColumn("rating",
                           F.col("rating_score").cast("double"))
             .withColumn("vote_count",
                           F.col("rating_count").cast("long"))
             .withColumn("genre", F.trim(F.col("genres"))))
cleaned = (prepared.dropna(subset=["rating"])
          .where(F.col("rating") > 0)
          .fillna({"genre": "未知类型", "country": "未知地区",
                   "directors": "未知导演", "summary": "暂无简介"}))
```

缺失值比例统计显示：rating_score 缺失 3,356 行，占 5.00%；genres 缺失 5,384 行，占 8.02%；directors 缺失 8,829 行，占 13.15%；summary 缺失 19,170 行，占 28.56%。此外，数据中还有 40,319 行 rating_score=0 的未形成有效评分记录。评分为分析的核心度量，空评分和零评分都不能被解释为真实的低评分，因此予以剔除；类型、国家、导演和简介属于描述性维度，以“未知”或“暂无简介”填充以保留有效评分样本。

清洗后获得有效评分影片 23,457 行，共移除 43,675 行非有效评分记录。清洗后评分均值为 6.5514，标准差为 1.3335，范围为 2.1–9.8；评分人数平均值为 13,029.53，最大值为 1,992,071。该结果比小样例更能反映真实数据长尾特征：大量影片没有形成评分，而高热度经典电影贡献了明显更高的投票规模。因此在 Top-N 查询中进一步增加评分人数阈值，避免极少量投票导致排名失真。

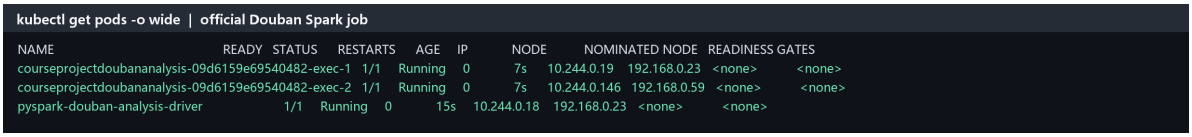


图 **21** 正式豆瓣分析作业运行时的 **Driver** 与两个 **Executor Pod**

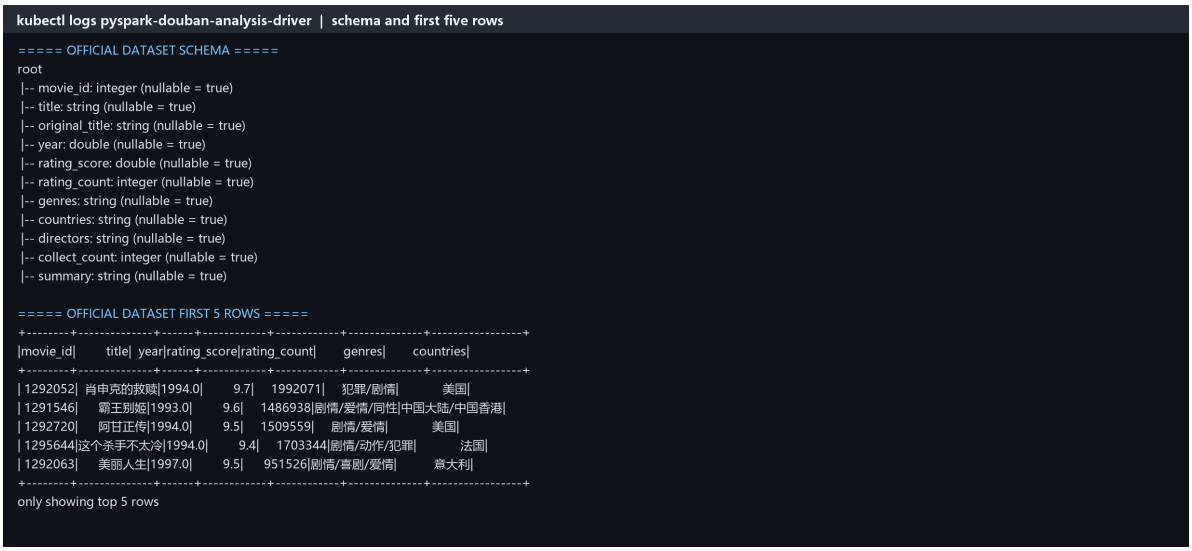


图 **22**　正式豆瓣数据集 **Schema** 与前 **5** 行运行输出

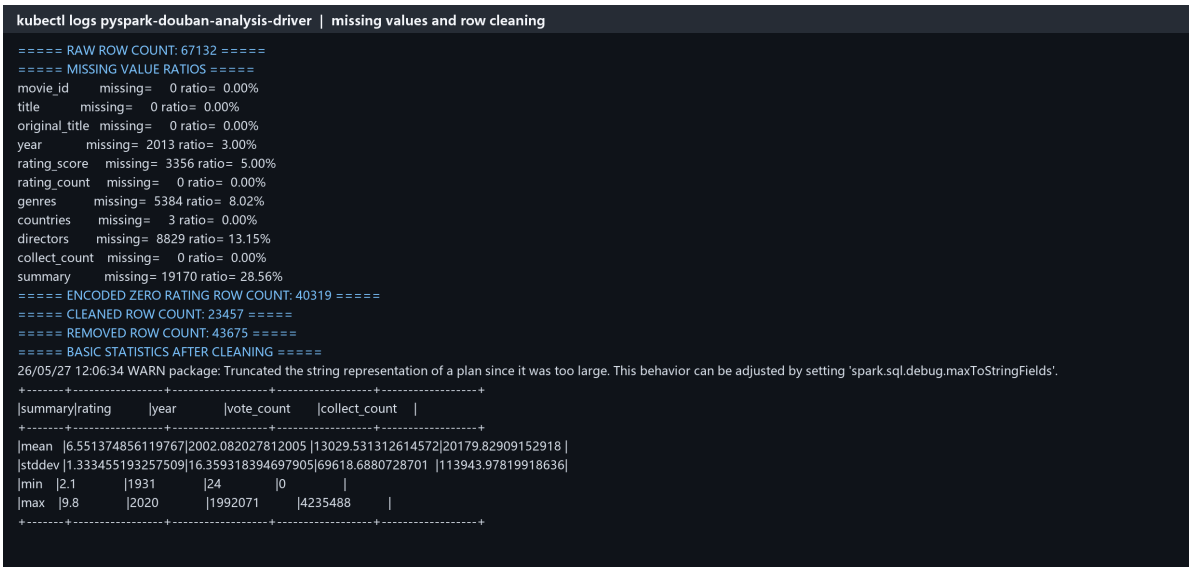


图 **23**　正式数据缺失值、零评分处理与清洗后统计输出

5.3 A-2 Spark SQL 统计分析

本实验在清洗后的 23,457 条有效评分记录上设计五类查询。由于一部影片可以同时属于多个类型，类型聚合、窗口排名和维表关联均先按 “/” 拆分 `genres` 并执行 `explode`，使每个影片–类型关系作为一个统计样本；Top-N 查询则保留影片粒度，并将有效热度阈值设为 `rating_count >= 100000`。

GROUP BY 查询显示，剧情类有效影片数量最多，为 11,996 部，平均评分 6.83，累计评分人数达到 180,350,722；喜剧和爱情分别为 5,260 部与 3,722 部，平均评分为 6.61 与 6.50。动画和传记的平均评分达到 7.24 与 7.27，但样本数量低于剧情类。这说明不能

表 **4**　**Spark SQL** 分析任务

查询类型	分析内容
GROUP BY 聚合	拆分类型后统计影片数、平均评分与累计评分人数，观察类型差异。
Top-N 排序	在评分人数达到 100,000 的影片中，按评分和人数选出前 10 名。
时间趋势	按年份统计有效影片数量和平均评分，分析近期年份变化趋势。
窗口函数	在五个主要类型内部使用 `row_number` 选出评分前三影片。
JOIN 分析	与类型维表广播 JOIN，聚合为叙事、商业、动画与纪录等类别。

只使用平均分比较类型质量，还应结合影片数量与累计评分人数判断结论的稳定性和受众覆盖程度。

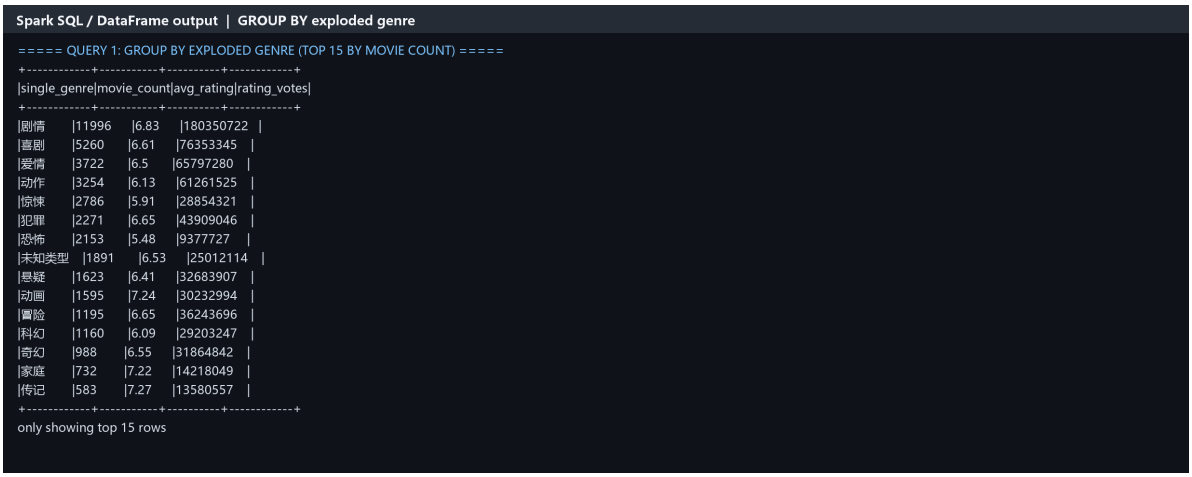


图 **24**　**GROUP BY** 查询：正式数据按类型聚合结果

Top-N 查询先以 100,000 人评价作为可信度门槛，再按评分降序和评分人数降序排序。结果中《肖申克的救赎》以 9.7 分和 1,992,071 人评价居首，《霸王别姬》和《控方证人》均为 9.6 分，前者拥有更大的评价样本。门槛处理降低了仅由少数观众打出高分的偶然影响，因此得到的列表同时具有评分表现和群体认可度，适合用作高口碑影片推荐的候选集合。



图 **25** **ORDER BY Top-N** 查询：高评分且高评价人数影片

时间维度查询选取 2015–2020 年观察近期变化。2015 年与 2016 年分别有 1,157 和 1,156 部有效评分影片，平均评分为 5.96 与 5.89；2017 年下降到 5.83，随后在 2018–2020 年回升至 5.98、6.00 和 6.04。值得注意的是，2020 年仅有 47 部有效记录，样本明显少于前几年，因此该年度均值只能作为当前数据截面的描述，不能直接推出整体影片质量上升的因果结论。



图 **26** 时间维度趋势查询：**2015–2020** 年有效影片统计

窗口函数查询在剧情、喜剧、动作、爱情和科幻五个主要类型内部执行 `row_number()`。剧情类第一名为《肖申克的救赎》，动作类第一名为《这个杀手不太冷》，科幻类前三名为《盗梦空间》《星际穿越》和《楚门的世界》。喜剧类出现评分 9.8 但评价人数仅 5,328 的条目，进一步说明窗口排名能保留明细并揭示低热度高评分的异常候选；若用于实际榜单，应继续叠加评分人数约束。

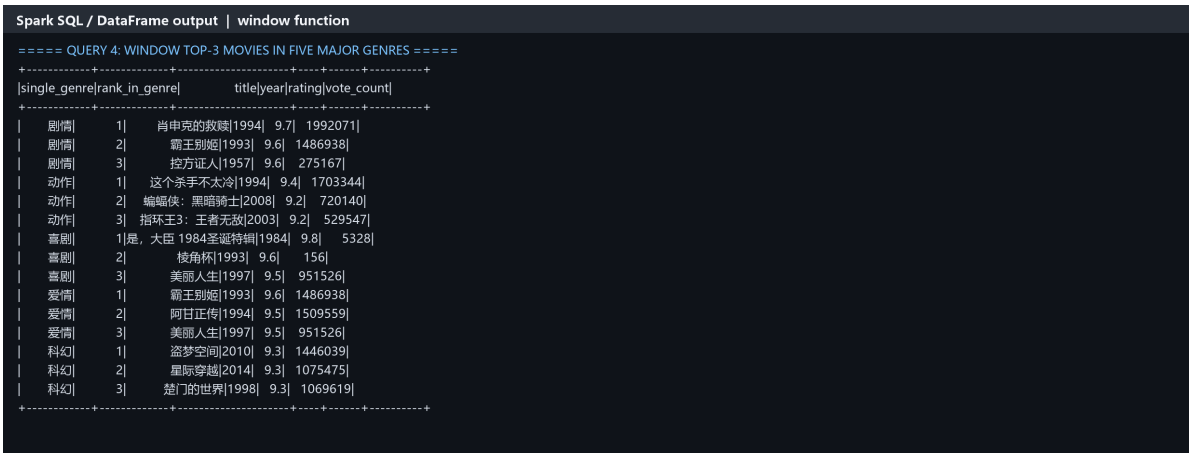


图 **27**　窗口函数查询：五个主要类型内部 **Top-3** 排名

JOIN 查询将拆分后的电影类型与小型类型维表执行广播关联，再汇总为更便于解释的业务组别。结果显示，“叙事与情感” 组包含 20,978 个类型关联记录，平均评分为 6.72；“类型与商业” 组包含 8,308 条，平均评分为 6.32；“动画与纪录” 组平均评分最高，为 7.27。该步骤对应真实数仓中事实表与维表关联场景，说明 Spark 不仅能做单表聚合，也能支持面向分析主题的维度重构。



图 **28**　**JOIN** 查询：类型维表映射后的分组统计

五类查询从总体分布、可信榜单、时间趋势、组内排序和维度建模五个角度分析同一份正式数据。结果中特别保留了样本量、累计评分人数和异常候选的讨论，避免只根据平均分作出过度推断，也说明 Spark SQL 查询结果需要结合数据质量与统计口径进行解释。

5.4 A-3 性能对比与 Amdahl 分析

性能对比选取 A-2 中的类型聚合查询，分别使用 Pandas 单机、PySpark 1 executor 和 PySpark 2 executors 实现。为避免仅比较缓存后的极短查询，本实验每轮计时均覆盖正式 CSV 读取、跨行解析、有效评分清洗、类型拆分和聚合排序全过程；每种模式连续执行 3 次，结果如表5所示。

表 **5**　性能对比记录表

执行模式	第 1 次/s	第 2 次/s	第 3 次/s
Pandas	1.007332	1.178784	1.205749
PySpark 1 executor	0.979194	0.726238	0.673658
PySpark 2 executors	1.114111	1.094048	0.887878

表 **6**　性能对比平均耗时与加速比

执行模式	平均耗时/s	标准差/s	相对 Pandas 加速比
Pandas	1.1306	0.1076	1.0000
PySpark 1 executor	0.7930	0.1634	1.4257
PySpark 2 executors	1.0320	0.1252	1.0956

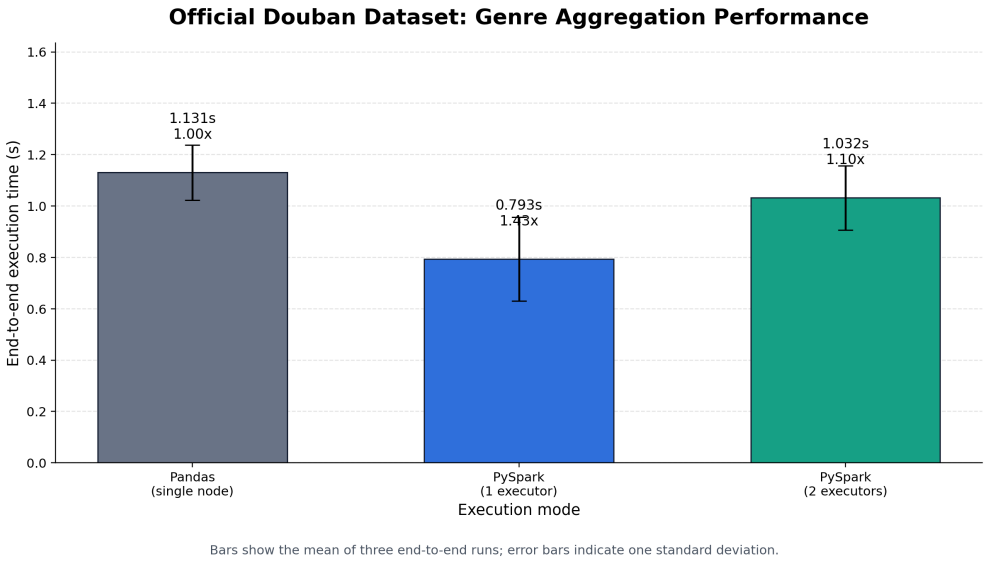


图 **29**　正式豆瓣数据集上 **Pandas** 与 **PySpark** 类型聚合耗时对比（误差线为标准差）

Amdahl 定律公式为：

$$S(p) = \frac{1}{(1-f) + \frac{f}{p}} \tag{1}$$

其中 $S(p)$ 表示使用 p 个并行单元时的理论加速比，f 表示程序中可并行部分比例。Amdahl 模型成立的前提是并行管理开销相对于有效计算足够小，而 Spark 的 Driver 调

度、Executor 启动、CSV 反序列化和按类型聚合产生的 Shuffle 都属于额外固定成本。

正式数据试验中，Pandas 平均耗时为 1.1306 秒，PySpark 1 executor 为 0.7930 秒，后者相对 Pandas 的加速比为 1.4257；但 PySpark 2 executors 平均耗时为 1.0320 秒，仅获得相对 Pandas 的 1.0956 倍加速，而且比 1 executor 慢 30.14%。以 1 executor 为基准，增加到 2 executors 的实测加速比为 $S_{2/1} = 0.7930/1.0320 = 0.7684 < 1$，因此不能将这一结果强行代入理想 Amdahl 公式反推出正的可并行比例。其根本原因是本实验原始文件仅约 39 MB，清洗后有效记录为 23,457 行，第二个 Executor 的调度、网络交换和 Shuffle 成本尚未被有效计算量摊薄。这一结果并非并行失败，而是量化说明了分布式计算的适用边界：在更大的 OBS 分区数据或多次复用的分析任务中，固定开销占比降低后，增加 Executor 才更可能带来接近理论值的收益。

六、 附加题实践

6.1 附加题 1：Prometheus 与 Grafana 监控系统

监控系统附加题使用教师离线包中的 `kube-prometheus-stack-83.7.0.tgz` 在 CCE 集群中部署 Prometheus 与 Grafana。实验目标不是单独启动一个可访问的页面，而是让 Prometheus 能够抓取 Kubernetes 集群、节点和 Pod 指标，并通过 Grafana Dashboard 展示节点 CPU 利用率折线图和各 Pod 内存使用柱状图。由于 CCE 集群中已有华为云自带的 `monitoring` 命名空间和部分监控组件，为避免资源名称冲突，本实验单独创建 `monitoring-course` 命名空间部署课程监控系统。

Listing 9: 监控系统部署命令

```
kubectl create namespace monitoring-course
kubectl create secret docker-registry swr-secret \
  --docker-server=swr.cn-south-1.myhuaweicloud.com \
  --docker-username=<SWR_USERNAME> \
  --docker-password=<SWR_PASSWORD> \
  -n monitoring-course

helm upgrade --install monitoring kube-prometheus-stack-83.7.0.tgz \
  -n monitoring-course \
  -f monitoring-values-czk.yaml \
  --skip-crds
```

部署过程中主要遇到两个问题。第一，集群中已经存在 Prometheus Operator 相关 CRD，直接安装教师 Chart 时出现 CRD 字段冲突。处理方式是先从离线 Chart 中提取 CRD，使用 `kubectl apply --server-side --force-conflicts` 更新 CRD，再在 Helm 安装时添加 `--skip-crds`，复用已更新的 CRD。第二，教师 Chart 中 node-exporter 默

认使用宿主机网络并占用 9100 端口，而 CCE 自带 node-exporter 已经使用该端口，导致课程版 node-exporter 调度失败。实验中将 `prometheus-node-exporter.hostNetwork` 和 `hostPID` 调整为 `false`，让 node-exporter 以普通 Pod 网络方式运行，最终两个 node-exporter Pod 均进入 `Running` 状态。

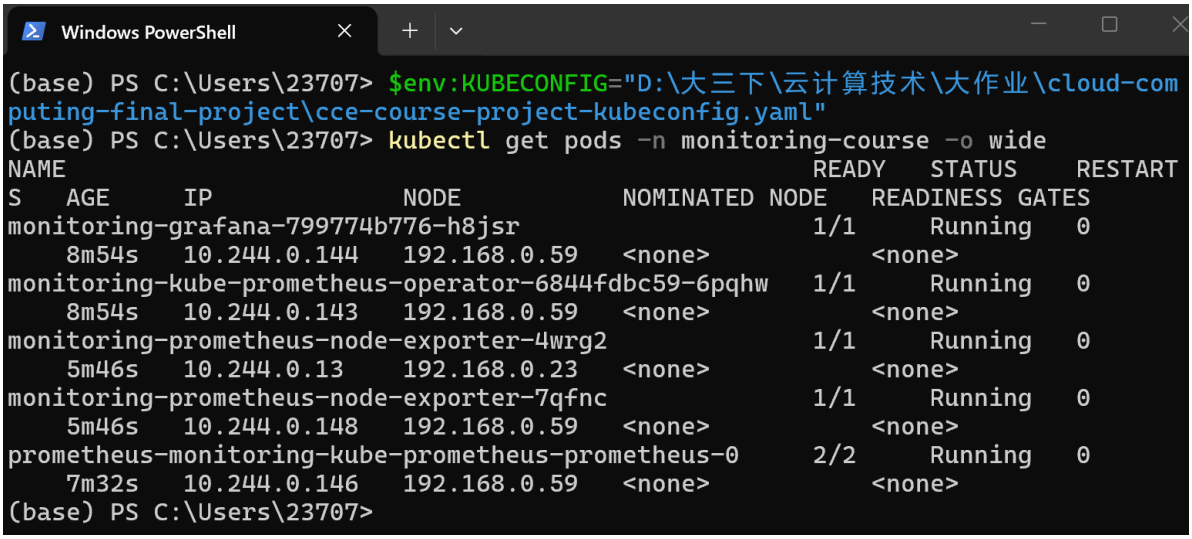


图 **30** **Prometheus**、**Grafana**、**Operator** 与 **node-exporter Pod** 均正常运行

Grafana 通过端口转发访问：

```
kubectl -n monitoring-course port-forward svc/monitoring-grafana 3000:80
```

进入 Grafana 后配置 Prometheus 数据源，地址为 `http://monitoring-kube-prometheus-prometheus.monitoring-course:9090`。Dashboard 中第一个面板使用如下 PromQL 计算节点 CPU 利用率：

```
100 - (avg by (instance) (irate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

第二个面板展示 Pod 内存使用情况，为避免原始字节值被错误显示为 TB，查询中将字节转换为 MiB：

```
topk(15, sum(container_memory_working_set_bytes{container!="",pod!=""}) by (pod) / 1024 / 1024)
```

Prometheus 采用 Pull 模式采集监控数据，即 Prometheus Server 按照配置的采集周期主动访问目标服务的 `/metrics` 接口并拉取指标，而不是由业务 Pod 主动上报。该模式的优点是采集周期、目标发现、失败重试和告警规则都集中在 Prometheus 侧管理，便于统一治理；缺点是被监控目标必须能被 Prometheus 网络访问，跨网络或边缘场景需要额外的代理或远程写入方案。Dashboard 中使用的 `node_cpu_seconds_total` 表示节点 CPU 在不同模式下累计消耗的时间，通过 idle 时间可以反推 CPU 利用率；`container_memory_working_set_bytes` 表示容器实际使用的工作集内存，适合观察

Pod 内存压力；up 指标表示 Prometheus 对目标的抓取状态，值为 1 代表抓取成功，值为 0 代表抓取失败。

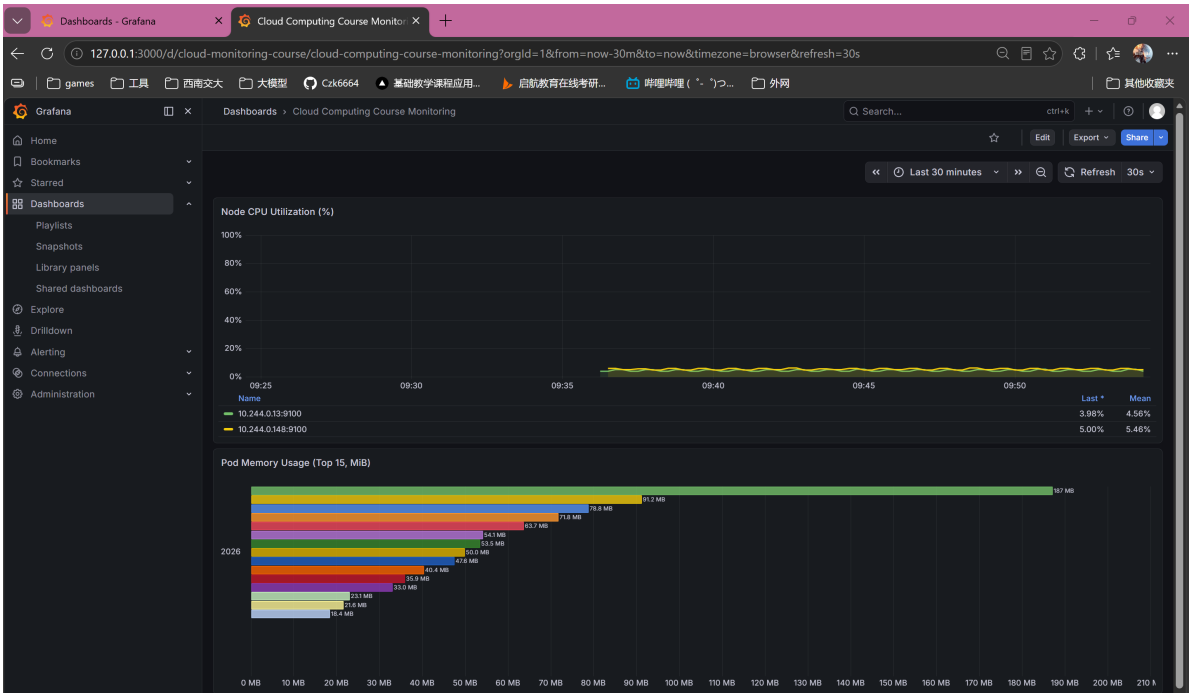


图 **31**　**Grafana Dashboard** 展示节点 **CPU** 与 **Pod** 内存指标

6.2 附加题 2：CI/CD 流水线

CI/CD 附加题基于 GitHub Actions 实现“代码提交、自动构建镜像、推送 SWR、更新 CCE Deployment”的端到端流水线。实验中首先将 `cloud-computing-final-project` 初始化为 Git 仓库，并在 `.gitignore` 中忽略 kubeconfig、日志、离线 Helm 包和缓存文件，避免将云账号凭据或本地运行产物提交到 GitHub。随后创建私有仓库 `Czk6664/cloud-computing-final-project`，将代码推送到 `main` 分支，并在 GitHub Actions Secrets 中配置 `SWR_USERNAME`、`SWR_PASSWORD` 和 `CCE_KUBECONFIG_B64`。

Listing 10: GitHub Actions 流水线核心步骤

```
name: Build Push Deploy
on:
  push:
    branches: [main]
  workflow_dispatch:

steps:
  - uses: actions/checkout@v4
  - run: echo "${{ secrets.SWR_PASSWORD }}" | docker login ...
  - run: docker build -f backend/Dockerfile -t $BACKEND_IMAGE:$TAG backend
  - run: docker build -f frontend/Dockerfile -t $FRONTEND_IMAGE:$TAG frontend
```

```
- run: docker push $BACKEND_IMAGE:$TAG
- run: docker push $FRONTEND_IMAGE:$TAG
- run: echo "${{ secrets.CCE_KUBECONFIG_B64 }}" | base64 -d > ~/.kube/config
- run: kubectl set image deployment/backend backend=$BACKEND_IMAGE:$TAG
- run: kubectl set image deployment/frontend frontend=$FRONTEND_IMAGE:$TAG
```

流水线运行时以当前提交号前 7 位作为镜像 Tag。本次成功运行的提交短 hash 为 3ba5771，Actions 依次完成源码拉取、SWR 登录、后端镜像构建、前端镜像构建、两个镜像推送、kubectl 安装、kubeconfig 写入、后端 Deployment 更新、前端 Deployment 更新和最终镜像验证。第一次由 push 自动触发的流水线失败，原因是仓库刚创建时 Secrets 尚未配置；完成 Secrets 后手动触发 `workflow_dispatch`，流水线在约 1 分 49 秒内全部通过。这个过程说明 CI/CD 环节不仅要关注 YAML 语法，还需要保证云端凭据、镜像仓库权限和 kubeconfig 都能被流水线环境正确读取。

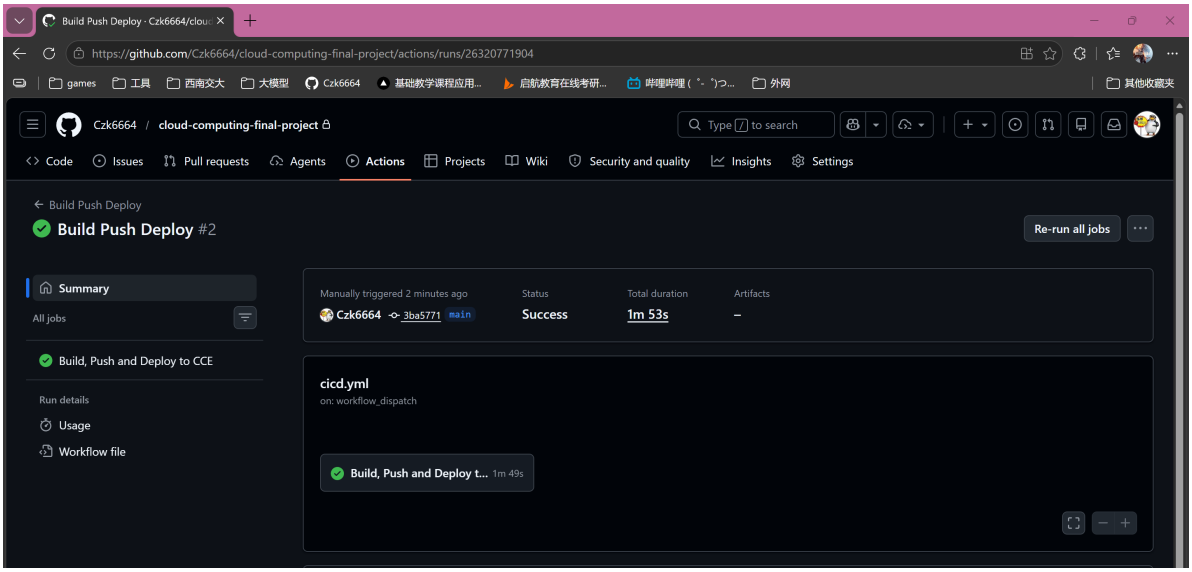


图 **32**　**GitHub Actions** 流水线运行成功

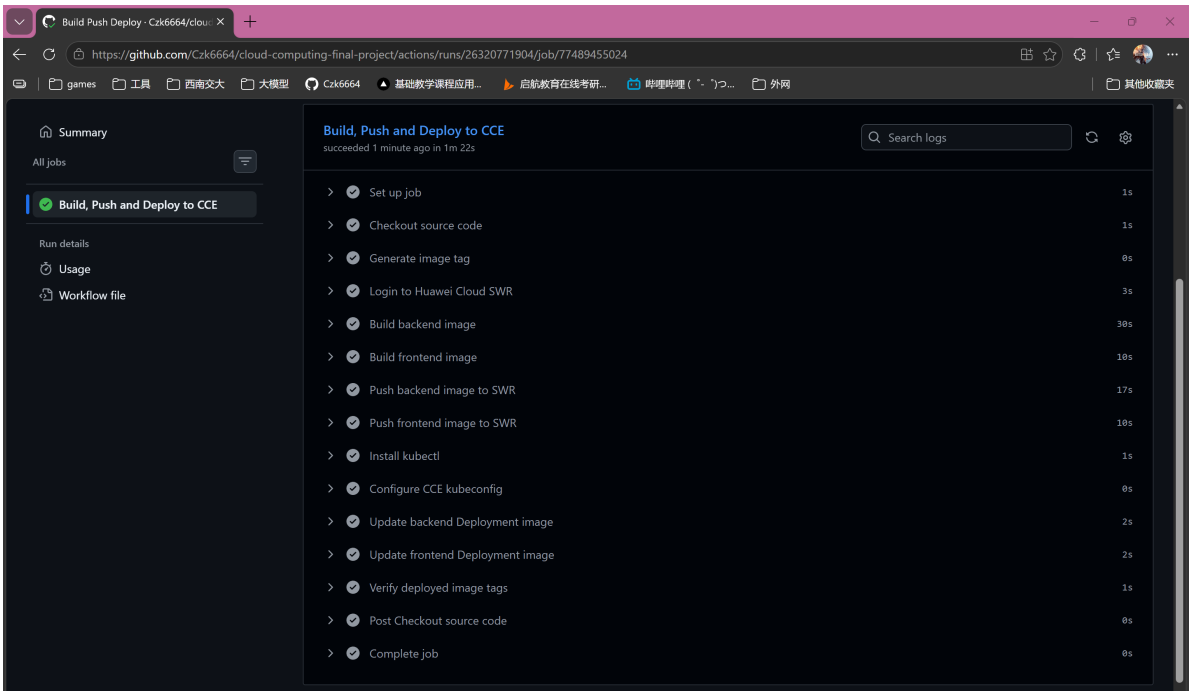


图 **33**　**CI/CD** 各阶段全部 **Passed**

集群侧验证命令如下：

```
kubectl get deployment backend frontend -n default \
  -o custom-columns=NAME:.metadata.name,IMAGE:.spec.template.spec.containers[0].image
```

输出显示后端与前端镜像均更新为 SWR 仓库中的 3ba5771 Tag，其中后端对应 backend:3ba5771，前端对应 frontend:3ba5771。这说明流水线不是只完成了镜像构建和推送，而是真正把新镜像发布到了 CCE 的运行态 Deployment 中。

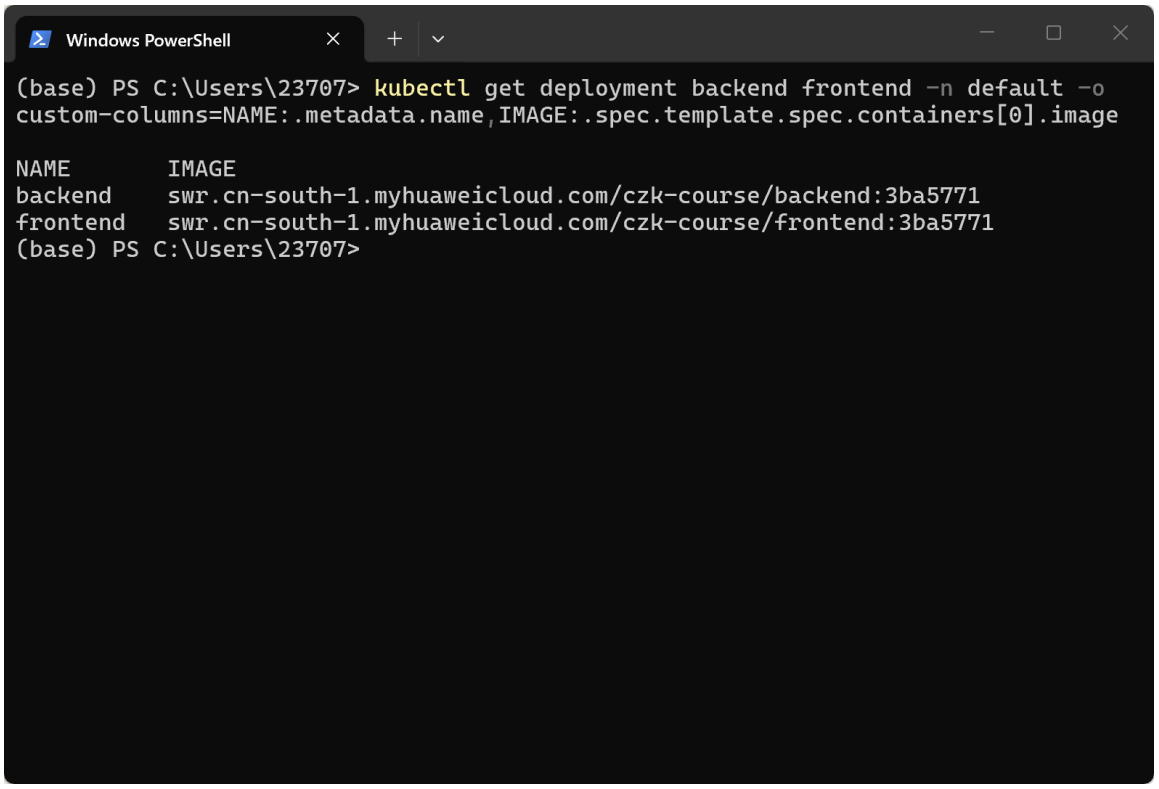


图 **34** **Kubernetes Deployment** 镜像 **Tag** 已自动更新为提交短 **hash**

持续集成 CI 主要关注代码提交后的自动构建、测试和制品生成，用于尽早发现依赖安装、镜像构建和集成阶段的问题；持续部署 CD 则在 CI 生成可信制品后，将镜像或应用自动发布到目标环境。本实验中，构建并推送 Docker 镜像属于 CI 到制品交付阶段，更新 Kubernetes Deployment 并等待滚动发布完成属于 CD 阶段。GitOps 的核心理念是以 Git 仓库作为系统期望状态的唯一来源，将应用代码、镜像版本和部署配置纳入版本管理，再由流水线或集群控制器将 Git 中声明的状态同步到实际集群。与手动修改集群相比，GitOps 具有可追踪、可回滚、可审计的优势，尤其适合多人协作和多环境发布。

6.3 附加题 3：边缘计算模拟：K3s + MQTT

前沿专题选择 C-2 边缘计算模拟方向，围绕“边缘侧轻量采集、云端统一接入、Redis 存储分析”的云边协同场景进行实践。实验目标是模拟边缘节点持续产生传感器数据，通过 MQTT 协议发送到云端 Kubernetes 集群中的消息代理，再由云端网关服务订阅消息并写入 Redis。虽然本机没有单独安装完整 K3s 虚拟机，但实验将本机 Python 传感器模拟程序作为边缘侧工作负载，CCE 中的 Mosquitto Broker、MQTT-to-Redis 网关和 Redis 作为云端侧服务，完整验证了从边缘数据发布到云端存储的链路。

实验架构由三部分组成。第一部分是边缘侧传感器模拟程序 `edge_sensor.py`，程序每隔固定时间生成一条 JSON 格式的传感器数据，字段包括设备编号、序号、温度、湿度、电压和时间戳，并发布到 `edge/sensors/edge-k3s-node-01/telemetry` 主题。第

二部分是在云端 CCE 集群中部署的 Mosquitto MQTT Broker，负责接收边缘侧发布的数据，并按照主题转发给订阅方。第三部分是云端 `mqtt-redis-gateway` 网关服务，该服务订阅 `edge/sensors/#` 主题，接收到消息后将原始 payload、topic、device_id、接收时间和 QoS 等字段写入 Redis Stream `edge:sensor:stream`，同时用 Redis String 保存每个设备的最新状态。

Listing 11: 边缘计算模拟部署与验证命令

```
kubectl apply -f edge/k8s/mqtt-broker.yaml
kubectl apply -f edge/k8s/mqtt-redis-gateway.yaml
kubectl port-forward svc/mqtt-broker 1883:1883
python edge/simulator/edge_sensor.py \
  --host 127.0.0.1 --port 1883 \
  --device-id edge-k3s-node-01 --count 10 --interval 0.3 --qos 1
kubectl logs deploy/mqtt-redis-gateway --tail=30
kubectl exec deploy/redis -- redis-cli -a redis2026 XLEN edge:sensor:stream
```

在镜像准备上，MQTT Broker 使用 `eclipse-mosquitto:2.0`，云端网关使用 Python、`paho-mqtt` 和 `redis` 客户端。为了避免 CCE 节点从 Docker Hub 拉取镜像失败，实验先将 Mosquitto 镜像转存到华为云 SWR，网关镜像也构建并推送到同一组织下，部署时分别使用 `eclipse-mosquitto:2.0` 和 `mqtt-redis-gateway:v1`。部署过程中曾遇到 Docker BuildKit 访问 Docker Hub 拉取 `docker/dockerfile:1` 前端超时的问题，因此删除 Dockerfile 第一行 `# syntax=docker/dockerfile:1`，改用普通 Dockerfile 构建流程，并继续使用清华 PyPI 镜像源安装 Python 依赖。

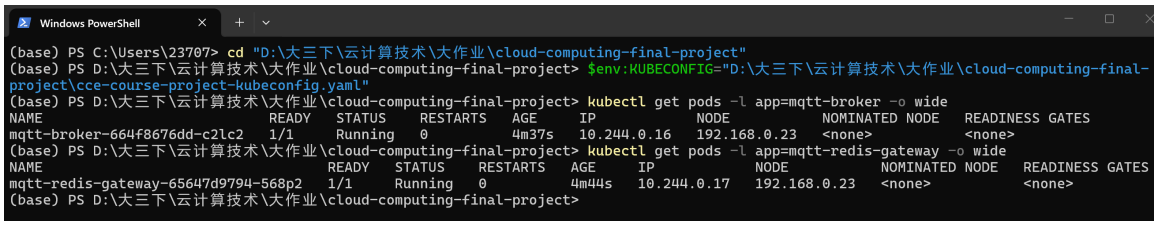


图 **35**　**MQTT Broker** 与 **MQTT-to-Redis** 网关 **Pod** 运行状态

运行边缘模拟器后，本机终端输出 10 条 `published` 记录，说明边缘侧已经通过 MQTT QoS 1 向云端 Broker 发送传感器数据。云端网关日志显示它成功连接 `mqtt-broker:1883`，订阅 `edge/sensors/#`，并逐条输出 `stored topic=edge/sensors/edge-k3s-node-01/telemetry`。Redis 验证结果显示 `XLEN edge:sensor:stream` 返回 10，`XRANGE` 可以看到 topic、device_id、payload、received_at 和 qos 字段，证明“边缘发布、MQTT 转发、云端消费、Redis 存储”链路完整运行。

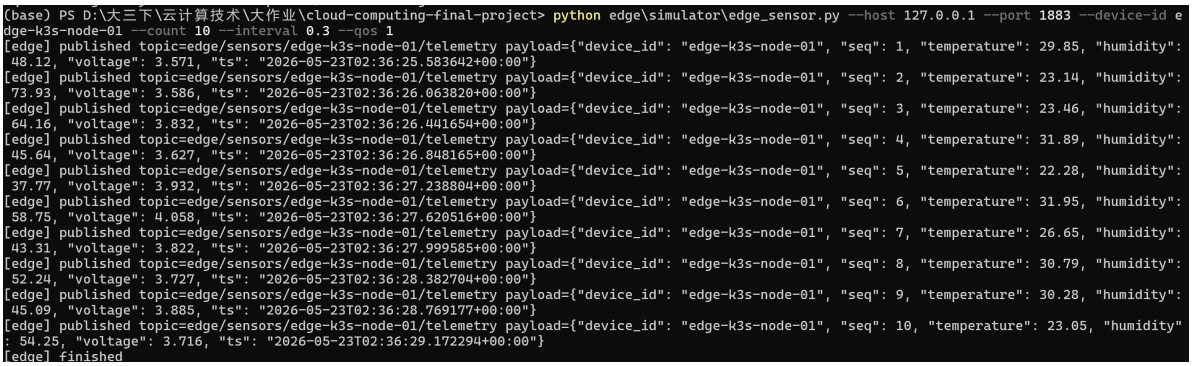


图 **36**　本机边缘模拟器发布传感器数据

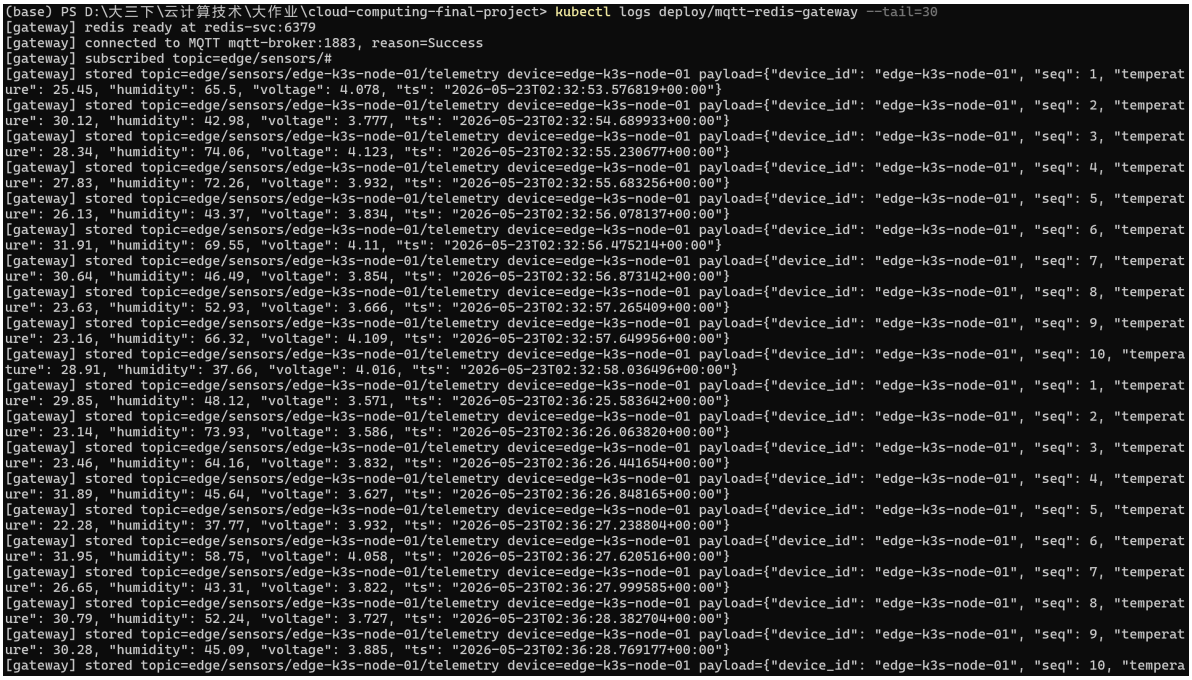


图 **37**　云端 **MQTT-to-Redis** 网关收到并存储消息

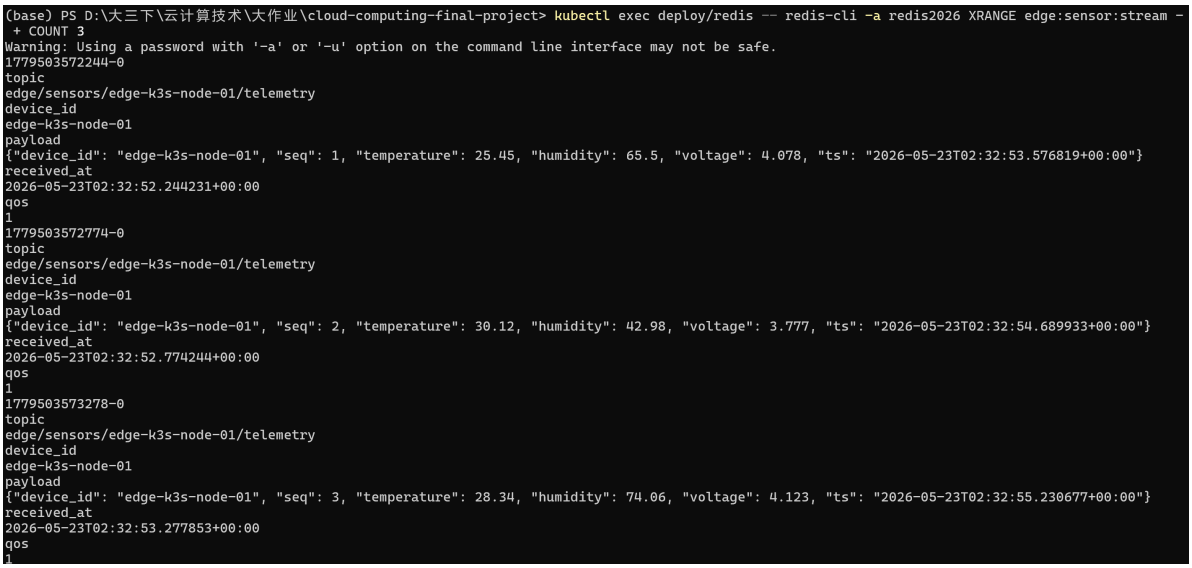


图 **38** **Redis Stream** 中保存边缘传感器数据

MQTT 的核心优势在于发布订阅模型和轻量级报文设计。与 HTTP 请求响应模式相比，MQTT 客户端不需要频繁建立连接，也不要求发布方知道具体消费者的位置。边缘设备只需要将数据发布到特定主题，后续由 Broker 完成消息路由，这种解耦方式适合设备数量多、网络质量不稳定、单个设备算力有限的场景。本实验使用 QoS 1 发布消息，含义是消息至少送达一次。相比 QoS 0，QoS 1 增加确认机制，可以在网络抖动时提高消息到达概率；相比 QoS 2，它避免了过高协议开销，因此适合传感器遥测这类允许少量重复、但不希望大量丢失的数据类型。

云边协同的挑战主要体现在延迟、可靠性和一致性三方面。首先，边缘节点到云端 CCE 的网络路径较长，公网链路可能带来几十毫秒到数百毫秒的额外延迟。如果业务是温湿度监控，这种延迟可以接受；但如果是工业控制或自动驾驶等实时场景，必须将决策逻辑下沉到边缘侧。其次，弱网环境下设备可能掉线，MQTT 可以通过长连接、QoS、保留消息和遗嘱消息改善可靠性，但不能完全替代本地缓存机制。实际生产中，边缘侧通常需要先将数据写入本地队列或轻量数据库，网络恢复后再批量补传。最后，云端 Redis 存储的是中心化视角的数据状态，边缘侧如果短时间离线，云端看到的状态就会滞后，因此系统需要在数据中同时保留设备时间戳和云端接收时间，区分“数据产生时间”和“数据入云时间”。

本实验的进一步思考是，边缘计算并不是简单地把所有业务都放到边缘，也不是把所有数据都实时上传到云。合理的云边协同应该根据业务时延要求和数据价值进行分层：低价值、高频率的原始数据可以在边缘侧先过滤或聚合；需要长期保存、跨设备分析和全局调度的数据再上传云端。MQTT 适合做轻量消息通道，但它本身不负责复杂计算、长期存储和数据治理，因此还需要 Redis、时序数据库、消息队列或流处理系统配合。本实验中 Redis Stream 只保存最近的遥测数据，适合作为课程实验和轻量缓存；如

果扩展到真实生产系统，可以进一步接入 Kafka、InfluxDB 或 Prometheus，实现更完整的时序分析和告警能力。

七、 关键文件与代码说明

本实验大作业工程位于 `cloud-computing-final-project/`。主要文件说明如下：

表 **7** 工程文件说明

文件或目录	作用
`backend/app.py`	Flask 后端 API，包含 `/api/ping` 和 Redis 访问接口。
`frontend/static/index.html`	Nginx 静态首页，展示验收身份信息。
`docker-compose.yml`	本地启动 backend、frontend 和 Redis 三个服务。
`k8s/*.yaml`	CCE 部署所需 Kubernetes 资源清单。
`spark/analysis.py`	正式豆瓣数据的 PySpark 清洗、SQL 查询与计时代码。
`spark/pandas_douban_performance.py`	Pandas 同口径类型聚合计时脚本。
`spark/sparkapplication-analysis-teacher.yaml`	CCE 上运行双 Executor 正式分析作业的资源清单。
`monitoring/monitoring-values-czk.yaml`	Prometheus 与 Grafana 离线 Helm 部署配置。
`.github/workflows/cicd.yml`	GitHub Actions CI/CD 流水线配置。
`edge/`	边缘计算模拟代码，包括 MQTT Broker YAML、网关服务和传感器模拟器。
`report/screenshot-checklist.md`	截图证据清单。

八、 总结与收获

本次课程设计将容器镜像、Kubernetes 编排、云负载均衡、持久化存储、配置分离、弹性伸缩和 Spark 并行计算串联成一个完整实践流程。第一部分实际完成了 2 个镜像的构建与推送，SWR 中 backend/frontend 均使用 `v1` Tag；CCE 集群包含 2 个 Worker 节点，节点状态均为 `Ready`，Kubernetes 版本为 `v1.34.3-r0-34.0.12.3`，高于任务要求的 1.27。应用部署后，后端 Deployment 最终以 2 副本运行，Redis 以 1 副本运行，后端 LoadBalancer 获得公网 IP `110.41.67.203`，访问 `/api/ping` 返回 HTTP 200 和 `{"status":"ok"}`，说明本地构建的服务已经完成从本机到云端的迁移。

从工程角度看，本实验最大的收获不是单条命令本身，而是对云原生资源之间关系的理解。Deployment 负责维持期望副本数，Service 负责服务发现和访问入口，ConfigMap 与 Secret 将非敏感配置和敏感配置分离，PVC 解决 Redis 这类有状态组件的数据持久化，HPA 则在指标系统可用后根据 CPU 利用率自动调整副本数。Redis 持久化验证中，PVC 容量为 10Gi，StorageClass 为 `csi-disk`，删除 Redis Pod 后 `testkey` 仍返回 `hello`，说明数据不依赖 Pod 生命周期。ConfigMap Volume 验证中，Nginx 配置文件修改为 `backend-svc:5001` 后能在容器内直接看到更新，截图完成后再恢复为 `5000`，体现了文件型配置与环境变量配置的适用差异。

实际排错过程也加深了对云平台运行机制的认识。Docker Desktop 崩溃、SWR manifest 解析失败、私有镜像拉取 401、LoadBalancer 长时间 pending、Pod 因 `Insufficient cpu` 无法调度、metrics-server Pending、HPA 初始指标为 `<unknown>` 等问题，都不是单纯代码错误，而是本地环境、镜像仓库、云厂商注解、资源请求和监控链路共同作用的结果。通过重新构建镜像、创建 `swr-secret`、补充 `elb.autocreate` 注解、降低轻量服务的 CPU request、启用并修正 metrics-server 资源配置，最终使系统恢复到可验收状态。HPA 压测中，由于本机没有 `ab`，改用 PowerShell 并发请求脚本；backend 副本从 1 个扩展到 4 个，CPU 指标最高达到约 `600%/60%`，停止压测后经过冷却窗口缩回 1 个，说明扩缩容链路已经生效。

第二部分 Spark 方向完成了 A-0 到 A-3 的完整闭环，并采用教师下发的正式 `douban_movies.csv` 重新执行分析。实验将离线 Spark Operator 与 PySpark 基础镜像推送至 SWR，再将正式数据和分析脚本封装为 `pyspark-analysis:douban-v4` 提交 CCE。运行过程中发现简介字段含跨行引号文本，默认 CSV 解析会将 67,132 条记录误解析为 117,156 条并产生不合理评分；加入 `multiLine` 与引号转义设置后，记录数恢复正确。正式数据中评分缺失 3,356 条、零评分 40,319 条，清洗后保留 23,457 条有效评分记录，评分均值为 6.5514，标准差为 1.3335。A-2 完成了 GROUP BY、可信热度门槛下的 Top-N、年份趋势、窗口函数和 JOIN 五类查询，所有结论均来自 CCE Driver 实际输出。

Spark 提交阶段还遇到既有应用和监控组件占用节点资源导致 Driver/Executor 因 `Insufficient cpu` 无法调度的问题。为不改变实验计算口径，我先保存运行状态，将

已完成验收的轻量服务和监控工作负载暂时缩容，按顺序完成 2 Executor 正式分析与 1 Executor 性能作业，再将应用与监控服务恢复。A-3 采用读取、清洗和类型聚合的端到端同口径计时：Pandas 平均耗时为 1.1306 秒，PySpark 1 executor 为 0.7930 秒，PySpark 2 executors 为 1.0320 秒。该 39 MB 数据集下单 Executor 最快，而双 Executor 比单 Executor 慢 30.14%，表明第二个执行器引入的调度与 Shuffle 开销尚未被有效计算量抵消；这一负加速结果比虚构线性提升更能准确体现 Amdahl 定律在小规模分布式任务中的适用边界。

附加题部分进一步把课程内容扩展到监控、自动化发布和边缘计算三个方向。监控系统使用教师离线 `kube-prometheus-stack` 部署 Prometheus 与 Grafana，最终在 `monitoring-course` 命名空间运行 5 个核心 Pod，并通过 Dashboard 展示节点 CPU 利用率和 Pod 内存使用。CI/CD 流水线使用 GitHub Actions 完成自动构建、推送 SWR 和更新 CCE Deployment，本次成功运行使用提交短 hash `3ba5771` 作为镜像 Tag，后端和前端 Deployment 均自动切换到对应镜像。边缘计算专题使用 MQTT 模拟边缘节点数据上云，边缘模拟器发送 10 条传感器数据，云端网关全部收到并写入 Redis Stream，`XLEN edge:sensor:stream` 返回 10，证明端到端链路可运行。三个附加题不仅补充分数项，也从可观测性、交付自动化和云边协同三个方面扩展了对云计算工程体系的理解。

总体来看，本实验从应用交付、平台部署、资源调度、数据持久化、弹性伸缩、并行计算、监控系统、CI/CD 和边缘计算多个层面展示了云计算技术的工程体系。报告整理过程中，我尽量让每一张截图都对应一个明确结论：Pod Running 证明资源创建成功，ELB 返回 `{"status":"ok"}` 证明对外访问成功，Redis 删除 Pod 后仍能读取数据证明持久化成功，HPA 副本变化证明指标驱动的弹性伸缩成功，Spark 查询输出证明数据分析逻辑成功，CI/CD 镜像 Tag 变化证明自动部署成功，MQTT 日志和 Redis Stream 证明云边数据链路成功。这种证据链比单纯堆叠命令更重要，也更符合课程设计对“能运行、能验证、能分析”的要求。

参考文献

[1] Apache Spark Project. Apache spark documentation. `https://spark.apache.org/docs/latest/`, 2026. Accessed for Spark SQL and PySpark concepts.

[2] Huawei Cloud. Huawei cloud cce documentation. `https://support.huaweicloud.com/cce/`, 2026. Accessed for CCE deployment concepts.

[3] The Kubernetes Authors. Kubernetes documentation. `https://kubernetes.io/docs/`, 2026. Accessed for Kubernetes resource concepts.